

Operaciones CRUD y gestión de transacciones

3. Operaciones CRUD y gestión de transacciones

Qué es CRUD

Vas a construir el CRUD de Libro como lo harías tú solo: empezando por la versión más simple, tropezando con sus problemas y resolviéndolos uno a uno.

CRUD	SQL	HTTP
Create	INSERT	POST
Read	SELECT	GET
Update	UPDATE	PUT (reemplazo completo); PATCH (parcial)
Delete	DELETE	DELETE

REST modela estas operaciones con los verbos que HTTP ya trae, en vez de inventar una ruta por acción: /libros/crear sería el estilo antiguo, no REST.

La primera pieza: el repository

```
public interface LibroRepository extends JpaRepository<Libro, Long> {}
```

T = Libro

La entidad que gestiona este repository.

ID = Long

El tipo de la clave primaria: el del campo @Id (en este curso, siempre Long).

Decenas de métodos

findAll(), findById(id), save(entidad), deleteById(id)... ya vienen heredados.

¿Cómo funciona una interfaz sin implementación? Al arrancar, Spring Data JPA genera una clase real que la implementa y la registra como bean. Tú nunca la ves: solo inyectas la interfaz.

Con JDBC «a pelo» escribirías cada consulta SQL a mano; aquí ese trabajo desaparece casi por completo.

Lo que ya tienes disponible, sin escribir nada

Método	Qué hace
<code>List<Libro> findAll()</code>	Todas las filas de la tabla libro.
<code>Optional<Libro> findById(Long id)</code>	Busca por clave primaria. Puede no encontrar nada.
<code>Libro save(Libro libro)</code>	Inserta si el id es null, o actualiza si ya existe.
<code>boolean existsById(Long id)</code>	Comprueba si existe, sin cargar la entidad.
<code>void deleteById(Long id)</code>	Borra la fila con ese id.
<code>long count()</code>	Cuenta las filas de la tabla.

El Optional de findById

```
libroRepository.findById(id)
    .orElseThrow(() ->
        new ResponseStatusException(
            HttpStatus.NOT_FOUND,
            "Libro no encontrado"));
```

«Puede que haya valor, puede que no»

Un Optional te obliga a decidir qué hacer si no se encuentra. `orElseThrow(...)` dice: si trae un libro, dámelo; si viene vacío, lanza esta excepción.

Y si necesitas una consulta que no viene de fábrica

Declaras el método, sin cuerpo, con un patrón de nombre

```
public interface LibroRepository
    extends JpaRepository<Libro, Long> {
    List<Libro> findByEditorialId(Long editorialId);
}
```

Equivale a

```
SELECT * FROM libro
WHERE editorial_id = ?
```

Patrón	Ejemplo	Equivale a
findByX	findByTitulo(String titulo)	WHERE titulo = ?
findByXAndY	findByTituloAndPrecioLessThan(...)	WHERE titulo = ? AND precio < ?
findByXOrderByY	findByEditorialIdOrderByPrecioDesc(...)	WHERE editorial_id = ? ORDER BY precio DESC
existsByX	existsByTitulo(String titulo)	como findByX, devuelve boolean
countByX	countByEditorialId(Long id)	como findByX, devuelve long

Es cómoda para consultas simples y fijas. Para filtros que cambian según el caso están las Specifications y @Query con JPQL, más adelante en este tema.

Primer intento: devolver la entidad tal cual

El service

```
@Service
@RequiredArgsConstructor
public class LibroService {
    private final LibroRepository libroRepository;

    @Transactional(readOnly = true)
    public Libro findById(Long id) {
        return libroRepository.findById(id)
            .orElseThrow(() -> new
                ResponseStatusException(
                    HttpStatus.NOT_FOUND,
                    "Libro no encontrado"));
    }
}
```

@Transactional(readOnly = true)

Solo consulta la base de datos, sin intención de modificarla.
Más adelante verás qué aporta.

El controller

```
@RestController
@RequestMapping("/api/v1/libros")
@RequiredArgsConstructor
public class LibroController {
    private final LibroService
        libroService;

    @GetMapping("/{id}")
    public ResponseEntity<Libro> findById(
        @PathVariable Long id) {
        return ResponseEntity.ok(
            libroService.findById(id));
    }
}
```

Controller → Service → Repository

El controller no toca el repository. Ese orden no cambia en todo lo que viene.

El problema: la entidad expone más de lo que quieres

GET /api/v1/libros/3 te devuelve algo así

```
{
  "id": 3,
  "titulo": "El nombre del viento",
  "precio": 19.95,
  "coste": 8.40,
  "fechaPublicacion": "2007-03-27",
  "editorial": {
    "id": 1,
    "nombre": "Plaza & Janés",
    "hibernateLazyInitializer": {}
  }
}
```

coste

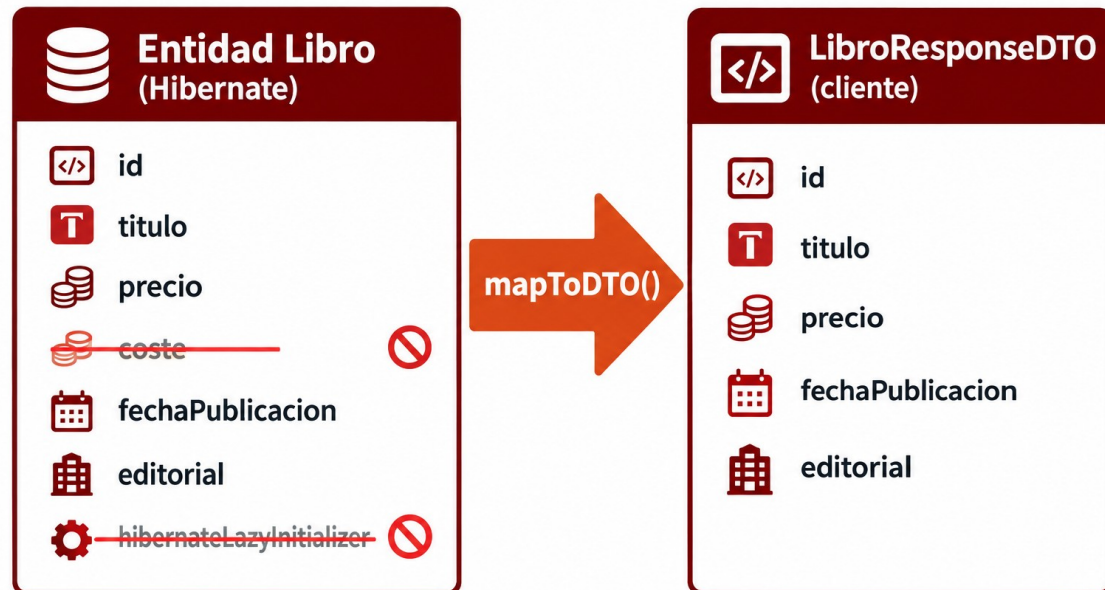
El precio de compra al proveedor: un dato interno que jamás debería llegar a un cliente. Cualquier campo private viaja tal cual al JSON.

hibernateLazyInitializer

Un detalle interno de cómo Hibernate representa la relación @ManyToOne con carga lazy, filtrado sin querer.

La entidad Libro está diseñada para que Hibernate la entienda, no para que la vea un cliente HTTP. Son dos propósitos distintos y una sola clase no puede servir bien a los dos.

La solución: DTOs, y el record de Java



Un DTO (Data Transfer Object) es un objeto hecho a medida de lo que sale, o entra, de tu aplicación.

Un record declara una clase inmutable en una línea

```
record Punto(int x, int y) {}
```

Lo que genera el compilador

Constructor, un getter por campo (punto.x(), no getX()) y equals/hashCode/toString. Campos final, sin setters: justo lo que necesita un DTO.

record no es @Entity

Libro y Editorial siguen siendo clases normales con @Getter/@Setter: necesitan ser mutables. Si ves record, es un DTO; si ves @Entity, es persistente.

El DTO de salida y mapToDTO

Solo lo que quieres que vea un cliente

```
public record EditorialDTO(
    Long id, String nombre) {}

public record LibroResponseDTO(
    Long id,
    String titulo,
    BigDecimal precio,
    LocalDate fechaPublicacion,
    EditorialDTO editorial
) {}
```

Ni rastro de coste

Ni de los detalles internos de Hibernate. Como coste no se copia en mapToDTO, desaparece del JSON.

findById traduce con mapToDTO (private: solo lo usa el service)

```
@Transactional(readonly = true)
public LibroResponseDTO findById(Long id) {
    Libro libro = libroRepository.findById(id)
        .orElseThrow(() -> new
            ResponseStatusException(
                HttpStatus.NOT_FOUND,
                "Libro no encontrado"));
    return mapToDTO(libro);
}

private LibroResponseDTO mapToDTO(Libro l) {
    EditorialDTO e = new EditorialDTO(
        l.getEditorial().getId(),
        l.getEditorial().getNombre());
    return new LibroResponseDTO(l.getId(),
        l.getTitulo(), l.getPrecio(),
        l.getFechaPublicacion(), e);
}
```

findAll, create y update llamarán al mismo mapToDTO. El controller solo cambia el tipo: ResponseEntity<LibroResponseDTO>.

Ahora la escritura: create()

Para crear necesitas un DTO de entrada. No vale reutilizar LibroResponseDTO: al crear, pedir un id no tiene sentido.

Un primer intento, sin pensarlo mucho

```
@Transactional
public LibroResponseDTO create(LibroCreateDTO dto) {
    Editorial editorial = editorialRepository
        .findById(dto.editorialId())
        .orElseThrow(() -> new
            ResponseStatusException(HttpStatus.NOT_FOUND,
                "Editorial no encontrada"));
    Libro libro = new Libro();
    libro.setTitulo(dto.titulo());
    libro.setPrecio(dto.precio());
    libro.setFechaPublicacion(dto.fechaPublicacion());
    libro.setEditorial(editorial);
    return mapToDTO(libroRepository.save(libro));
}
```

El DTO de entrada

```
public record LibroCreateDTO(
    String titulo,
    BigDecimal precio,
    LocalDate fechaPublicacion,
    Long editorialId) {}
```

Datos basura

Nada impide mandar titulo vacío, precio -5, fecha 2099 o editorialId -1: create() no comprueba nada.

Bean Validation: rechazar datos basura

Anotaciones sobre el propio DTO

```
public record LibroCreatedDTO(  
    @NotBlank String titulo,  
    @NotNull @PositiveOrZero BigDecimal precio,  
    @NotNull @PastOrPresent LocalDate fechaPublicacion,  
    @NotNull @Positive Long editorialId  
) {}
```

@Valid en el controller las activa

```
@PostMapping  
public ResponseEntity<LibroResponseDTO> create(  
    @Valid @RequestBody LibroCreatedDTO dto) {  
    return ResponseEntity.status(HttpStatus.CREATED)  
        .body(libroService.create(dto));  
}
```

Falta una dependencia

spring-boot-starter-webmvc no trae Bean Validation. Añade spring-boot-starter-validation (Hibernate Validator): sin ella, @NotBlank y compañía ni existen en el classpath.

Un 400 antes de tu código

@RequestBody convierte el JSON al DTO; @Valid comprueba las restricciones. Si alguna falla, Spring responde 400 Bad Request sin ejecutar nada.

La familia de Bean Validation

Anotación	Qué exige
@NotNull	No puede ser null (precio, fechaPublicacion, editorialId).
@NotBlank	No null, ni vacío, ni solo espacios (titulo).
@NotEmpty	Igual, para colecciones y arrays.
@Size(min=, max=)	Longitud dentro de ese rango.
@Positive / @PositiveOrZero	Mayor que cero / mayor o igual (editorialId / precio).
@Negative / @NegativeOrZero	Lo mismo, en negativo.

Anotación	Qué exige
@Min(valor) / @Max(valor)	El número no baja de / no supera ese valor.
@Past / @PastOrPresent	Fecha anterior a ahora / anterior o igual (fechaPublicacion).
@Future / @FutureOrPresent	Fecha posterior a ahora / posterior o igual.
@Email	Forma de dirección de correo.
@Pattern(regex = ...)	Coincide con la expresión regular dada.

Todas se pueden combinar sobre el mismo campo, como @NotNull @PositiveOrZero en precio.

update() y delete(): el recurso que no existe

update() reutiliza LibroCreateDTO, pero antes carga el libro

```
@Transactional
public LibroResponseDTO update(Long id,
    LibroCreateDTO dto) {
    Libro libro = libroRepository.findById(id)
        .orElseThrow(() -> new
            ResponseStatusException(HttpStatus.NOT_FOUND,
                "Libro no encontrado"));
    Editorial editorial = editorialRepository
        .findById(dto.editorialId())
        .orElseThrow(() -> new
            ResponseStatusException(HttpStatus.NOT_FOUND,
                "Editorial no encontrada"));
    libro.setTitulo(dto.titulo());
    libro.setPrecio(dto.precio());
    libro.setFechaPublicacion(dto.fechaPublicacion());
    libro.setEditorial(editorial);
    return mapToDTO(libroRepository.save(libro));
}
```

delete(): nada que cargar, comprobación ligera

```
@Transactional
public void delete(Long id) {
    if (!libroRepository.existsById(id)) {
        throw new ResponseStatusException(
            HttpStatus.NOT_FOUND,
            "Libro no encontrado");
    }
    libroRepository.deleteById(id);
}
```

Sin orElseThrow

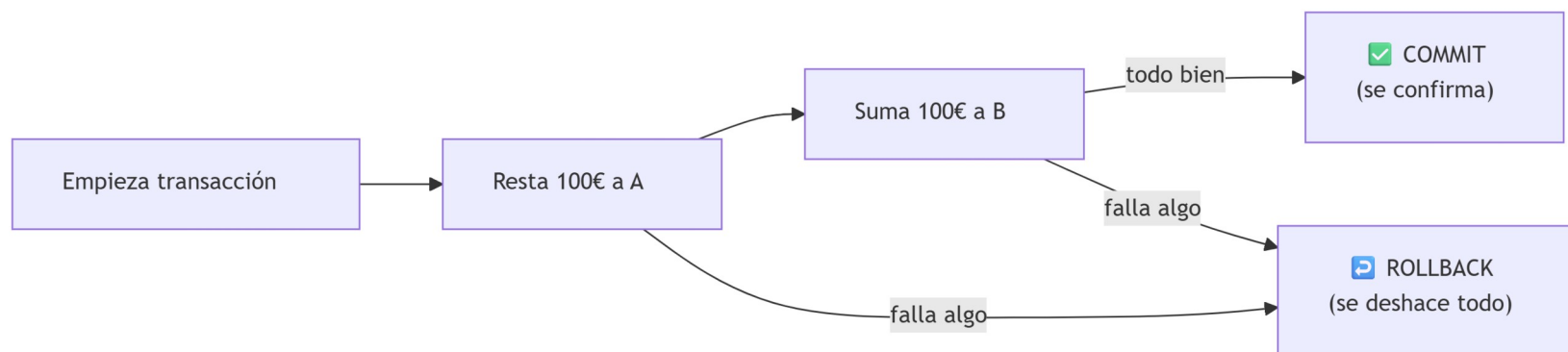
Podrías acabar usando un null: compila, pero falla al ejecutar.

findAll() no tiene el problema

Una lista vacía no es un error. Usa readOnly y convierte cada entidad a su DTO.

Transacciones: todo o nada, y ACID

Como una transferencia bancaria: restar 100 € de una cuenta y sumarlos a otra son dos operaciones, y si falla justo después de restar, el dinero desaparece. Una transacción las agrupa: todo o nada.



Atomicidad

Se ejecuta entera, o no se ejecuta nada.

Consistencia

Pasa de un estado válido a otro, nunca a uno a medias.

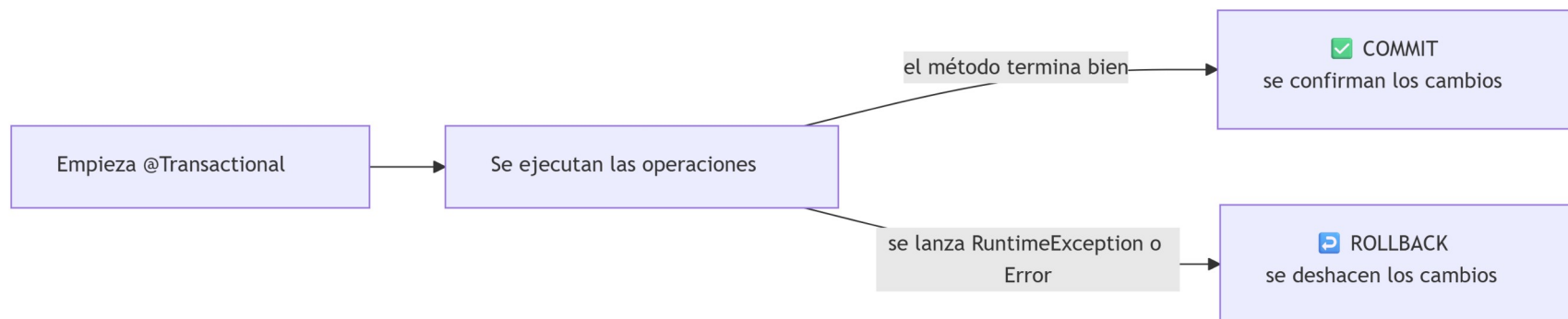
Aislamiento

Dos transacciones a la vez no se interfieren.

Durabilidad

Confirmada (commit), sobrevive aunque el sistema se caiga.

Todo esto lo gestiona Spring: @Transactional



Abre la transacción

Antes de ejecutar el método.

Commit automático

Si el método termina correctamente.

Rollback automático

Si termina con una
RuntimeException (p. ej.
ResponseStatusException).

```
@Transactional(rollbackFor = IOException.class)
public void importarLibros() throws IOException {
    // ...
}
```

Excepciones comprobadas

No hacen rollback por defecto: indícalo con rollbackFor. En JDBC puro harás commit y rollback a mano.

Transacciones de solo lectura

```
@Transactional(readonly = true)
public LibroResponseDTO findById(Long id) {
    // ...
}
```

Solo se espera consultar

Permite optimizaciones a Spring y a Hibernate y deja clara la intención del código.

Operación	Anotación
Consultar datos	@Transactional(readonly = true)
Crear, actualizar o eliminar	@Transactional

Esa es la regla de todo el CRUD: readOnly para leer, @Transactional a secas para escribir.

Las piezas juntas: el LibroController completo

```
@RestController
@RequestMapping("/api/v1/libros")
@RequiredArgsConstructor
public class LibroController {
    private final LibroService libroService;

    @GetMapping
    public ResponseEntity<List<LibroResponseDTO>>
        getAll() {
        return ResponseEntity.ok(
            libroService.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<LibroResponseDTO>
        getById(@PathVariable Long id) {
        return ResponseEntity.ok(
            libroService.findById(id));
    }
}
```

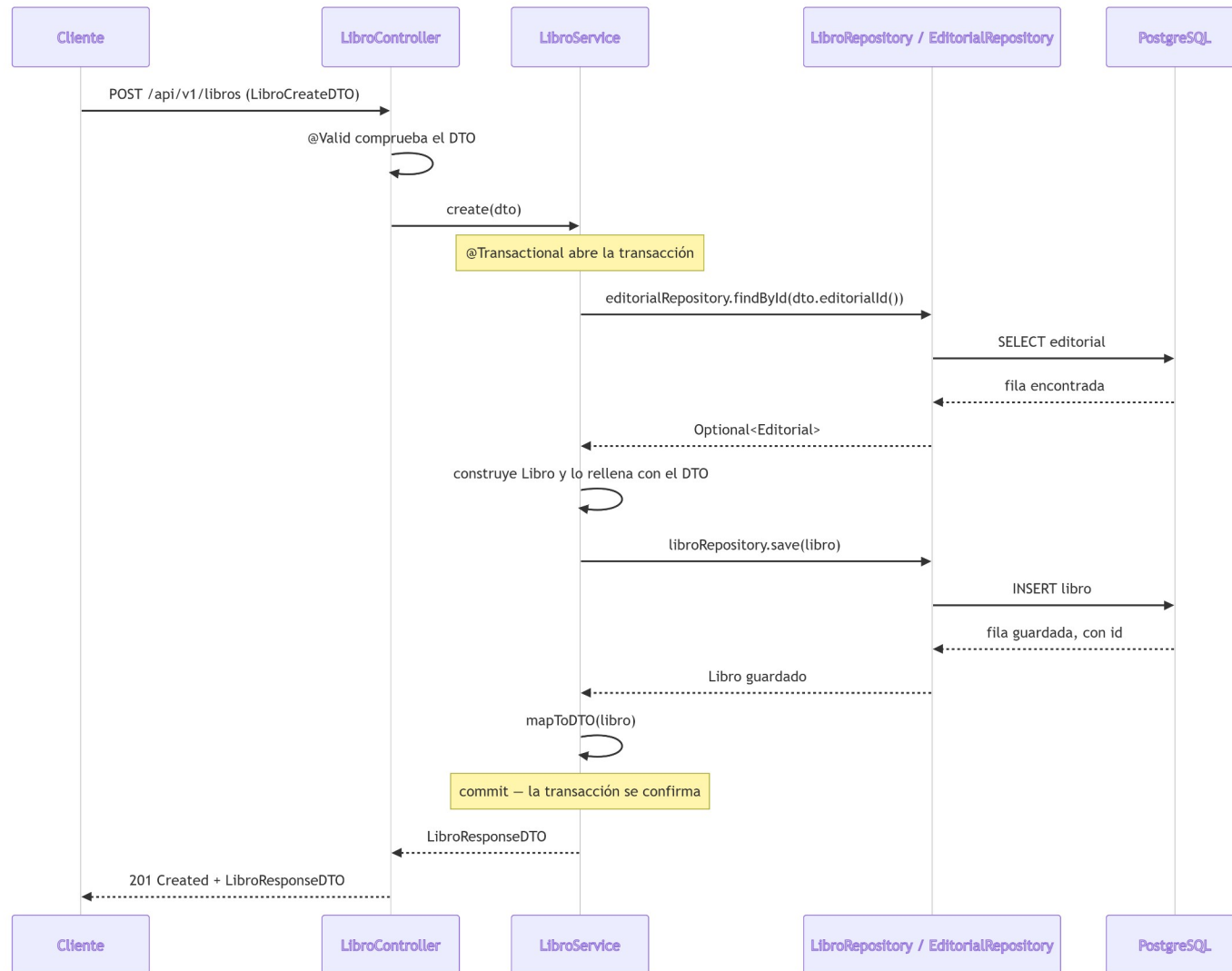
```
@PostMapping
public ResponseEntity<LibroResponseDTO> create(
    @Valid @RequestBody LibroCreateDTO dto) {
    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(libroService.create(dto));
}

@PutMapping("/{id}")
public ResponseEntity<LibroResponseDTO> update(
    @PathVariable Long id,
    @Valid @RequestBody LibroCreateDTO dto) {
    return ResponseEntity.ok(
        libroService.update(id, dto));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete(
    @PathVariable Long id) {
    libroService.delete(id);
    return ResponseEntity.noContent().build();
}
}
```

create responde 201, update 200 y delete 204 sin cuerpo. El controller nunca habla con el repository ni construye SQL: solo traduce entre HTTP y LibroService. Con esto ya puedes abordar la Actividad 1.2.

El viaje completo de un POST /api/v1/libros



De principio a fin

Todas las piezas del apartado juntas: desde que llega la petición hasta que el cliente recibe el 201.

Siempre el mismo flujo

Controller → Service → Repository, cada uno hablando solo con el siguiente.

Ideas clave

1

CRUD = Create/Read/Update/Delete. En HTTP: POST, GET, PUT o PATCH, y DELETE.

2

Un repository (extends `JpaRepository<X, Long>`) da `findAll`, `findById`, `save`, `existsById` y `deleteById` sin escribir SQL.

3

Métodos sin cuerpo con un nombre convencional (`findByEditorialId...`) generan consultas; para filtros dinámicos, `Specifications` y `@Query`.

4

Devolver la entidad tal cual expone campos internos: se convierte a un DTO. Un record es una clase inmutable en una línea.

5

Bean Validation (`@NotBlank`, `@PositiveOrZero...`) con `@Valid` rechaza datos basura con un 400 antes de ejecutar tu código.

6

«No encontrado» se resuelve con `orElseThrow` o `existsById` lanzando `ResponseStatusException(NOT_FOUND)`.

7

Una transacción es una unidad todo o nada: `commit` si todo va bien, `rollback` si algo falla. ACID la garantiza.

8

`@Transactional` lo gestiona Spring; `readOnly = true` marca la lectura. El flujo es siempre `Controller` → `Service` → `Repository`.